Yanjin Li

Self-taught AI engineering & research · product and finance background

I'm a self-taught builder — high agency, deep curiosity, and the courage to keep trying. My path runs from studying French and Japanese to a Master's in Management & Finance at **ESCP** (Paris & London); and from strategy at **TikTok** and product operations at **DiDi** to Global Markets at **HSBC** and corporate banking at **BNP Paribas** — across Beijing, London and Paris.

Since early 2026 I've taught myself AI entirely from papers and online courses, then built **11 projects in three months** to make each idea real — from training-method research (a **4.16B-parameter** model trained with no backpropagation) and a schema-first cross-device AI system (**42 JSON schemas**), to trustworthy applied ML and shipped local-first products. I want to take on harder problems here, alongside people who are great at it.

geraldineliyanjin@gmail.com   github.com/Yanjin-ai   yanjin-ai.github.io/portfolio   Painting · @yanjin.l.painting

---

INDEX — 11 PROJECTS

Grouped by type. Click any entry to jump to its full write-up.

A · Research & Training Methods

Presented as condensed technical reports: objective → method → staged progression → results → lessons.

B · AI Systems & Architecture

Presented by architecture: data model, schemas, codebase, core mechanisms, design decisions.

05	**Gemma4all**	Cross-device task system; schema-first, event-sourced, local-first, HITL approvals.	`system`
06	**VoiceInput**	macOS push-to-talk speech input; pure Swift, zero dependencies.	`system`

C · Applied ML & Decision Systems

Presented as ML systems: data, model, calibration/uncertainty/fairness, and the operational decision layer.

07	**TriageGeist**	Emergency-triage decision stack; label-leakage finding, conformal uncertainty, fairness audit.	`applied-ml`
08	**Maze Crawler**	Layered multi-unit strategy agent; BFS planning, conflict resolution, CI.	`applied-ml`

D · AI Products & Tools

Presented as products: positioning, what it does, how it was built, current status.

09	**AI Showrunner**	One-sentence → subtitled vertical short film; VLM critic loop, character consistency.	`product`
10	**Offline French Companion**	In-browser LLM language tutor; dictionary + speech + OCR + TTS, fully offline.	`product`
11	**Atelier Post**	Shipped art-to-Instagram tool; one codebase across web, PWA, iOS.	`product`

A · RESEARCH & TRAINING METHODS

01 Zero Gradient RESEARCH

Python · Kaggle T4 · 2026-06-25 → 07-05

Trained a **4.16B-parameter content-routed Mixture-of-Experts language model with backpropagation globally disabled** on a single 16 GB T4, then ran a five-phase program to map exactly what such a model can and cannot learn — and used a controlled experiment to prove the limiting factor is architectural, not the absence of gradients.

SCALE 4.16B params, 0.42% experts updated/step **RESULT** test ppl 1355 (vs 159 same-budget backprop control)

ARTIFACTS paper draft · 8-pg workshop paper · arXiv package

OBJECTIVE

Test whether layer-local learning rules (no end-to-end gradients) can scale to a multi-billion-parameter LM under a hard single-GPU memory constraint, and characterize the resulting capability ceiling precisely.

METHOD

Layer-local losses with depth-supervised readout heads replace gradients; EMA-prototype content routing with capacity control; frozen causal attention (reservoir-computing style). 4 layers × 950 experts, top-2 routing, BPE-32k, WikiText-103. Fully deterministic and reproducible. A same-budget backprop transformer (42M) serves as the cost control.

PROGRESSION

A–D Build & scale — nano prototype → 4B on T4. Found the working triangle: extreme-sparse update budget (0.42%/step) + content routing + depth supervision. Reached ppl 1355; measured the 3–9× cost vs the backprop control (159).

E Find the boundary — post-training on three task types: sentiment 79%, NLI at chance, arithmetic at chance. A clean, structure-driven boundary.

F–G In-model fixes — 5 improvement routes (data / objective / attention / non-collapse readout / deep BP); none broke the boundary. Three mechanistic hypotheses falsified by probes.

H Controlled study — an isolated standard trainable-attention transformer solves the same synthetic tasks at 100% and reaches 69.97% on real SNLI (vs 33.4%). Bottleneck located in the frozen architecture.

CONCLUSION

The capability ceiling is architectural, not algorithmic: frozen attention cannot represent relational structure, regardless of gradient budget. "Capability boundary as a function of task structure" — bag compositionality ≪ relational alignment ≪ multi-step depth — is an axis orthogonal to scaling laws.

LESSONS

Ran it like a real research program: pre-registered pass/fail gates, hypothesis→probe→falsification loops, LOCKED/INTERPRETATION/PROPOSAL labels separating fact from inference. Caught and publicly corrected two false-positive results caused by training artifacts, then standardized on task-agnostic closed-form probes for all capability claims.

LIMITATIONS

1355 ppl is far from a usable LM (effective only on bag-like tasks and LM); control models are 12–21M, so their 100% transfer isn't directly extrapolable to 4B (stated in the paper); even the control hits a multi-step wall at k≥4 and scores 2% on generative GSM8K.

`PyTorch (grad disabled)` `layer-local learning` `MoE · 950 experts/layer` `EMA prototype routing` `frozen attention` `WikiText-103 · BPE-32k`

02 NeuroGolf — ARC-AGI Circuit Compiler RESEARCH

Python · ONNX / onnxruntime · 2026–07

Reframed an ARC-AGI "neural golf" competition — minimize the size of a static ONNX network that reproduces each of 400 ARC tasks, scored `max(1, 25 − ln(params + memory))` — from a learning problem into **program synthesis + circuit golfing**, by reverse-engineering the scorer into a cost model and compiling known task rules into minimal, overfitting-proof integer-weight circuits.

SCOPE 400 ARC tasks · one ONNX circuit each **BASELINE** strongest public bundle LB 7243 (3-mo community-tuned)

DELIVERABLE transferable cost-model + methodology note

REFRAMING

The competition looks like few-shot learning, but an official per-task Python generator (arc-gen) *is* the ground-truth rule in executable form — and the same-distribution sampler for the hidden private set; a symbolic DSL solver (arc-dsl) covers 280 tasks. So the task is not "infer the rule" but "compile a known rule into a minimal, constraint-satisfying, overfitting-proof ONNX circuit." Identifying this was the single largest lever.

METHOD

Built the toolchain: an oracle replicating the official scorer; an `onnx.helper` graph-building library with a static-cost function and single-node circuit factories; a three-level validator (official examples + N freshly generated examples + constraint audit, run in isolated subprocesses); and auto-fitters (single-Conv integer-weight LP, two-layer torch).

FINDINGS

cost model Reverse-engineered the scorer into "physics": cost = params + intermediate-tensor bytes; input/output tensors and node attributes are free (single-node circuits ≈ 0 memory); the log scale means redesign ≫ tuning (halving cost is only +0.69). This drove every design choice.

anti-overfit Because the private set is generator-sampled, fit and validate on ≥3000 freshly generated examples — the official ~265 examples give false positives.

key lesson An LP-fit conv passed 3000 local examples yet scored 0 on Kaggle: LP found weights separating the current samples, not the true rule. Conclusion — fitted circuits are private-set-untrustworthy; only hand-written integer-weight circuits that implement the true rule (double-sided margin ≥ 1) are robust.

audit Reverse-audited the scorer for exploitable holes (early >9000 exploit scores had been re-scored away by the organizers) and confirmed the top ~8000 is legitimate golfing with no residual exploit.

CONCLUSION

The strongest public baseline (LB 7243) is a 3-month community-tuned bundle already near the physical cost floor on many tasks; the realistic legal ceiling is ~7300–

7500, and top-10 (~7920) needs hundreds of tasks of deep golfing. Marginal gains come only from structure the baseline didn't cover, which needs genuine spatial computation (enclosure / ray-cast / counting) that resists cheap circuits.

LESSONS

- When a contest is dressed as "few-shot learning," first check for a generator / reference implementation that degenerates learning into compilation — the biggest lever.
- Treat any minimization contest's scoring function as physics: measure the exact cost of each op / dtype / structure, draw a design ladder, and select by it — never by intuition.
- Distinguish fitting a function that passes the samples from implementing the rule that produces them. With an oracle available, always implement the true rule — the only thing robust to a private distribution.

STATUS

Net Kaggle gain over the strong baseline was small (task192 +1.0, task243 +0.19, several tasks robustified to net ~0). Two ORT platform bugs cost real time (group-Conv tiling; same-process session pollution — solved with subprocess evaluation). The int8/QLinearConv two-layer route (the correct solution for ~10 non-linearly-separable tasks) was designed but unfinished. The durable deliverable is the transferable technical note, written to carry over to ARC-AGI-2.

`ONNX · onnxruntime` `program synthesis` `scorer reverse-engineering` `integer-weight circuits` `LP / torch fitting` `subprocess batch eval`

03 Nemotron Post-training Pipeline RESEARCH

Python · A800 80GB · 2026-06-10 → 06-15

A reproducible post-training pipeline for NVIDIA's **Nemotron-3-Nano-30B** (MoE + Mamba–Transformer hybrid) reasoning model. The LoRA fine-tune failed; the most valuable output is a **fully root-caused failure** with a diagnosed cause, six postmortem documents, and a designed recovery route.

BASELINE rule-based solver EM 0.7516 (100% format) **INFRA** 30B MoE in 4-bit on one GPU (40 GB)

OUTCOME SFT loss 0.33, generation collapsed to EM 0%

OBJECTIVE

Post-train a 30B reasoning model to produce boxed CoT answers in the competition format, via a rule-teacher → CoT-SFT → preference-learning pipeline, on a single cloud GPU.

WHAT I BUILT

- A deterministic rule-based solver as teacher (EM 0.7516, 100% format) — clean distillation labels instead of noisy model outputs.
- Loaded the 30B MoE in 4-bit on one GPU; found and monkey-patched a `cache_position=None` bug in the Nemotron-H integration; wrote a masked SFT collator (loss on response tokens only).
- Multi-dimensional reward shaping (correctness + format + reasoning + length) with automatic preference-pair construction for GRPO/DPO.

FAILURE & DIAGNOSIS

SFT loss converged to 0.33, but autoregressive generation collapsed into repetition (EM 0%). Root cause: a teacher-forcing / generation mismatch in the Unsloth + hybrid-architecture path. NeMo-RL integration was abandoned for environment complexity (CUDA13 / mcore).

LESSONS

Six postmortem documents; a designed recovery route (native PEFT distillation with forced reasoning prefixes and a post-trained teacher). The permanent process rule I took away: never launch a 500-step run before sanity-checking the full train→generate loop on 5 samples / 50 steps.

`Transformers · PEFT · Unsloth · TRL` `4-bit quantization` `masked SFT collator` `GRPO reward shaping` `upstream bug patch`

04 Orbit Wars RESEARCH

Python · Kaggle env · 2026-06-18 → 06-22

A structured multi-agent game player for a Kaggle simulation. The substance is not the agent but the **evaluation methodology**: an interpreter-faithful world model, bilateral self-play, style-diverse opponent pools, and an honest study of why offline metrics failed to predict the live ladder. Named honestly as **heuristic search with rigorous evaluation, not deep RL**.

METHOD

- World model ported function-by-function from the official interpreter, verified by unit tests (rotation prediction error < 0.5 units over 10 turns) — planning runs on real physics, not approximations.
- Evaluation harness most competitors skip: both-sides play to cancel positional bias, four opponent styles via config perturbation to avoid single-mirror overfitting, full round-robin Elo arenas.
- Coordinate-descent parameter search over the pooled opponents.

KEY FINDINGS

- Parameter interaction: one setting scored −56 Elo in isolation but +108 when searched jointly — single-parameter ablation is misleading.
- Offline / online validity gap: +108 Elo offline coincided with a *drop* on the live ladder. I documented the discrepancy and kept every rejected experiment in the codebase with its Elo delta, rather than reporting the flattering number.

CONCLUSION

Parameter search plateaued around mid-ladder; breaking through needs new capability (lookahead / simultaneous-move search), not more tuning. The transferable lesson is about evaluation validity: an offline harness can be internally rigorous and still not predict the real objective.

`interpreter-faithful physics` `unit-tested world model` `bilateral self-play Elo` `4-style opponent pool` `coordinate-descent search`

05 Gemma4all `SYSTEM`

TS · Python · Swift · Kotlin · 2026-05-16 → 07-07

A cross-device task system: submit a natural-language task on your phone; a routing engine decides which device (phone / desktop / cloud) executes it based on privacy, capability and load; execution hands off across devices with checkpoints; every state change is an immutable event. Built on four principles — **task-first, schema-first, control-plane-first, local-first**.

PROTOCOL 42 JSON Schemas · 7 protocol docs + ADRs **SURFACE** 5 codebases (TS/Py/Swift/Kotlin)

INFERENCE LiteRT-LM on Metal GPU, 4–6s/task

ARCHITECTURE

A TypeScript/Node control plane owns all state (SQLite, repository pattern, AJV validation at every boundary, 7 API route groups). A Python asyncio desktop runtime executes tools. Native hosts on iOS (SwiftUI), Android (Compose/Hilt) and macOS (menu-bar) discover each other over Bonjour. On-device inference via LiteRT-LM with Metal GPU dispatch (one engine, per-task conversations).

DATA MODEL

Three-layer object model: Device → Task → Run, with Event as the only state-change entry point. All types are generated from 42 JSON Schemas (Draft 2020-12) — zero hand-written DTOs across four languages. A HandoffPayload carries checkpoint + tool history + context with idempotency keys, so a task started on the phone finishes on the Mac.

CORE MECHANISMS

- Event sourcing — Task/Run state can only change via `EventService.processStateTransition`; no route handler mutates state directly, giving a complete, replayable audit history.
- Routing engine — hard-filters on privacy boundary (local_only / private_lan / cloud_ok), soft-ranks on device state; the same task can legally run in three places and the choice is explained via events.
- Fail-closed security (added as a dedicated hardening phase) — central security policy, catastrophic-pattern screening, human-in-the-loop approval gate that denies when no approval UI is reachable.

DESIGN RATIONALE

Protocol-first because one spec driving four languages removes most integration friction for a solo builder, and an assistant that touches your files/calendar must be auditable, fail closed, and keep data local by default. An ADR records why JSON Schema was chosen over Zod / Protobuf / OpenAPI (multi-language + LLM-readable + mature type generation).

STATUS

Production-shaped two-runtime system (mobile + desktop) end-to-end in ~1 month; the cloud runtime and web console are specified but stubbed. Android is thinner than iOS; TS unit-test coverage lags the Python side (16 tests); event delivery is polling (WebSocket is v2).

`TypeScript/Node · Express` `Python asyncio` `JSON Schema 2020-12 · AJV` `LiteRT-LM · Metal GPU` `SwiftUI · Compose · Bonjour` `SQLite · event sourcing`

06 VoiceInput `SYSTEM`

Swift · macOS 14+ · 2026-05-23

A macOS menu-bar utility: hold `Fn`, speak, release — real-time speech is transcribed and injected into any text field. **Pure Swift, zero third-party dependencies**, with latency engineering that makes dictation feel instant.

CORE MECHANISMS

- Global `Fn` monitoring via CGEventTap with an NSEvent fallback for OS-version resilience; Accessibility + Microphone + AppleEvents permission handling.
- AVAudioEngine capture with Float32 → PCM-16 linear-interpolation resampling (44.1/48k → 16k); streaming over a WebSocket implementing an OpenAI-Realtime-compatible protocol (DashScope Qwen3 ASR).

DESIGN DECISIONS

- Pre-roll buffering: recording starts the instant `Fn` goes down and is cached while the socket connects, then flushed — collapsing perceived latency to well under typical connect time.
- Seamless Chinese-IME handling: detect an active Chinese input method, switch to ABC for injection, restore afterwards — solving a real, unsolved-by-default macOS pain point. 7 languages.

STATUS

Usable (Makefile one-command build, diagnostic scripts). Cloud ASR, so it needs network and a paid API key; macOS-only; young project without longitudinal users.

`Pure Swift · zero deps` `CGEventTap + NSEvent` `AVAudioEngine resampling` `WebSocket streaming ASR` `menu-bar app`

C · APPLIED ML & DECISION SYSTEMS

07 TriageGeist `APPLIED-ML`

Python · 2026-06-19 → 06-21

An emergency-department triage decision stack. Mid-competition I found that **99.7% of chief-complaint phrases mapped monotonically to a single triage label** — the dataset rewarded a lookup table, not a model. I documented the leak, declined to exploit it, and instead built a text-blind physiological model wrapped in calibration, conformal uncertainty, a fairness audit, and an operations console. Three days.

MODEL text-blind multi-task GBDT, QWK 0.9287 **CALIBRATION** ECE −46% (isotonic)

TESTS 12/12 (incl. leakage guards)

DATA & FEATURES

Leakage-safe 4-table joins at patient level; physiology-grounded features (NEWS2, shock index, GCS, missingness indicators) through a single `transform()` shared by training and serving — no train/serve skew by construction.

DECISION STACK

- Model — multi-task GBDT (acuity + admission + length-of-stay), QWK 0.9287; threshold tuning lifts acuity-4 recall without hurting critical recall.
- Governance — isotonic calibration (ECE −46%); distribution-free conformal sets that defer ~89% of cases to human review (itself evidence the labels are physiologically underdetermined); entropy-ranked review queues.
- Fairness — bootstrap 95% CIs (not point estimates) surfacing a 3× undertriage gap for Estonian speakers and a QWK collapse (0.78) when pain scores are missing, flagged as data-quality red flags rather than tuned away.
- Operations — a clinician-readable priority equation (transparent weighted terms, not a black box), resource buckets, ED-load forecasts, and a 4-view dashboard (Queue / Patient / Ops / Audit) with human-override logging.

JUDGMENT

A triage system trained on a complaint lookup table would be dangerous in a real ED; what a hospital needs is trustworthy uncertainty — which ~10% can be safely auto-triaged (0.16% undertriage in that slice) and which must reach a nurse. I also tried a physiology "safety-net" rule that lowered QWK and reported it anyway, because it was evidence about the labels, not a failure to hide.

LIMITATIONS

Synthetic competition data — no prospective or multi-site validation (TRL 3–4, explicitly not a medical device). Fairness gaps are quantified but unresolved; acuity-4 remains the hardest class; priority weights are hand-tuned.

`LightGBM · multi-task` `isotonic calibration` `conformal prediction (APS)` `bootstrap fairness CIs` `shared transform()` `4-view dashboard`

08 Maze Crawler APPLIED-ML

Python · Kaggle env · 2026-06-16

A layered multi-unit strategy agent for a Kaggle competition — **parser → world state → BFS planner → role policies → conflict resolver** — built, evaluated, and put under CI in a single day.

ARCHITECTURE

- Single-responsibility layered modules; every unit proposes a move, and a 3-round iterative resolver detects collisions, lets priority win, and re-plans losers — legality guaranteed by construction.
- All 77 strategy thresholds centralized in one config module — the whole policy is tunable without touching logic; phase switching (opening / economy / survival) driven by turn count and map pressure.
- Cross-turn memory of tiles, mines and enemy tracks reduces re-exploration under fog of war.

RESULT

16-seed evaluation: mean reward 168.5 vs 0 for the random baseline. Shipped with a Makefile, batch-eval scripts and GitHub Actions CI. Honestly a v1 heuristic baseline — parameters hand-tuned rather than systematically searched; the test suite is light.

`BFS planning under fog` `3-round conflict resolution` `77 centralized params` `cross-turn memory` `Makefile + CI`

D · AI PRODUCTS & TOOLS

09 AI Showrunner PRODUCT

Python · FastAPI · Qwen · 2026–06 → 07–06

An end-to-end AI production pipeline for vertical short films: one sentence in, a subtitled 9:16 video out — **script → storyboard → per-shot video generation → a vision-model critic that reviews each shot and drives targeted retries → multi-language subtitles**.

POSITIONING

A virtual-production workflow for short-drama studios (TikTok / Reels / 小红书): compress the script-to-cut cycle from days to hours, keep characters consistent across shots, and localize to multiple languages from one script.

HOW IT'S BUILT

- A closed-loop critic, not a one-shot pipeline: each generated clip is sampled at 3 frames and scored by a VL model on narrative alignment, character consistency and technical quality; failures produce *targeted* fix-prompts (e.g. "face blurred → add sharpness cues") within a retry budget.
- Character consistency across shots: canonical reference portraits locked into a library and injected as image-to-video first frames (verified consistency 9–10).
- Every stage is a replayable Pydantic JSON artifact (story bible → shot specs → QA reports → edit list); human-in-the-loop gates at outline / storyboard / final cut; per-run cost ceilings.

STATUS

6 complete end-to-end films produced (incl. a 26-second short). FastAPI dashboard with live event stream. TTS/lip-sync designed but unbuilt; the final hackathon run hit API-quota limits; cloud-deploy scripts written but not field-verified.

`FastAPI · asyncio` `Pydantic JSON artifacts` `Qwen–Max planner` `Qwen–VL critic` `I2V/T2V generation` `ffmpeg · multi–lang SRT`

10 Offline French Companion PRODUCT

WebLLM · vanilla JS · PWA

A full French-learning assistant that runs **entirely inside the browser** — LLM dictionary, bilingual speech input, photo OCR, and spoken pronunciation — with no server, no API, and no network required.

POSITIONING

Privacy-first, offline language learning: the model *is* the app. A learner picks a model tier once, then looks up words, dictates, or photographs a menu — data never leaves the device.

HOW IT'S BUILT

- WebLLM runs a quantized Qwen (0.5B / 1.5B / 3B tiers, 350MB–1.7GB); a <2KB system prompt plus JSON-schema-constrained output turns a small local model into a structured dictionary (IPA, senses, conjugation, etymology, quotations), with graceful fallback to free text on parse failure.
- Three input modalities converge on one inference path: typed text, long-press bilingual speech (Whisper-tiny on WebGPU), and photo OCR (Tesseract.js) of menus and signs; Web Speech API for pronunciation.
- A Service-Worker cache strategy engineered around GB-scale weights: network-first for app code, cache-first for immutable CDN model shards, deferring to the engine's own cache to avoid double-storing gigabytes.

STATUS

Production-quality PWA (installable, works offline). 3B-tier entries are occasionally imperfect (cross-check serious queries); WebGPU browser coverage is still maturing; the first download needs patience on Wi-Fi.

`WebLLM · quantized Qwen` `Whisper-tiny · WebGPU` `Tesseract.js OCR` `Web Speech TTS` `PWA · Service Worker` `vanilla JS`

11 Atelier Post `PRODUCT · SHIPPED`

vanilla JS · Capacitor · 2026-06-09 → 07-05

A **shipped** tool that turns artists' work into polished Instagram content — single posts, carousels, and video stories — from **one vanilla-JS codebase running as website, PWA and iOS app**. Live on the web today.

POSITIONING	A privacy-first, backend-free posting studio for artists and photographers: upload → adjust → lay out → export → share, entirely local, with no tracking and no upload.
HOW IT'S BUILT	• A hand-built Canvas 2D engine: perspective (keystone) correction via triangular-mesh warping, 5-layer photo adjustment, Ken Burns motion for video slideshows, and 3×3 mural splitting exported in Instagram upload order. • Local-first storage in IndexedDB; one codebase spans web, PWA and iOS (WebView) via Capacitor, with no platform-specific hacks; 8 UI languages. • Three workbenches — Artwork / Series / Story. I also built an AI caption feature, evaluated it, and removed it — deciding what to cut is part of product judgment.
STATUS	Live on GitHub Pages (artworktopost); 29 commits over 27 days; iOS build with App Store assets prepared. The single-file codebase (~1.6k lines JS) is nearing its maintainability boundary; no automated tests; monetization gates scaffolded but dormant.

`vanilla JS · single file` `Canvas 2D engine` `IndexedDB · local-first` `Capacitor iOS` `GitHub Pages · live`

PROFESSIONAL EXPERIENCE

Markets & corporate banking, then product & growth.

BNP Paribas — Global Corporate Banking Intern — Paris · Dec 2025 – Present

- Structured tailor-made financial solutions with coverage, product, legal and risk teams for international clients, focused on transactional services (cash management, FX hedging, short-term financing), delivering competitive offers under tight deadlines alongside front-office sales and traders.
- Built and presented pitchbooks for high-value clients, combining market-trend analysis with pricing scenarios and cross-border liquidity solutions; contributed to commercial strategy and product placement for global corporate clients.
- Assisted in executing complex deals: performed country-risk analysis (notably in emerging markets), participated in legal and credit-term negotiations, and optimized risk-mitigation tools (guarantees, supply-chain finance).
- Worked closely with Relationship Managers to identify and pitch cross-selling opportunities (FX, investment solutions, structured cash products) and ran seasonal client analysis focused on the TMT and Electronics sectors.
- Supported front-office operational efficiency by automating reporting dashboards and monitoring key client flows, in regular contact with middle and back offices.
- Helped land internal AI tooling — built RAG knowledge bases on Open WebUI and Dify to improve client-response efficiency — for a book including CATL, BYD, Alipay, Midea, TCL, ANTA and JD.com.

HSBC — Global Markets, Liquidity & Investment Intern — Paris · Jun – Nov 2024

- **Treasury / macro / rates analysis** — monitored central-bank rates (Fed / ECB / BoE) and money markets to optimize short-term EMEA portfolios; worked with treasury on liquidity allocation under LCR/NSFR constraints and curve positioning.
- **Pricing & modeling** — developed dynamic pricing models for wholesale deposits (benchmark calibration, internal treasury-curve integration); collaborated with trading desks.
- **Portfolio optimization** — enhanced EMEA short-term rate portfolios using macro signals and liquidity constraints; built automated VBA/Excel tools tracking rate moves and pricing gaps in real time to identify arbitrage.
- **Governance & reporting** — managed money-market statistical reporting; contributed to the ALCM committee (gap-risk analysis, funding-cost optimization, sensitive-position tracking); automated reports via VBA.

TikTok — **Growth Strategy Intern** Beijing · Feb — May 2023

- Contributed to 10+ strategic projects, increasing SaaS/AI market share by 23% through market-trend and competitive-landscape analysis.
- Analyzed financial and operational data in SQL and Excel, generating weekly reports for a 20+ member team to enable optimal resource allocation and improve ROI.
- Led process optimization — created new online channels with design, product and IT teams, raising user conversion by 13% in one month.
- Coordinated cross-functional project management to deliver strategic initiatives aligned with business objectives.

Uber China · DiDi — **Product Management & User Operations Intern** Beijing · Aug 2022 — Jan 2023

- **Business-operations optimization** — managed 3 projects to optimize operations, using modeling and data analysis to develop concrete strategies for user interaction and cost reduction.
- **Data & process analysis** — analyzed large datasets and optimized processes in Python, SQL and VBA, identifying optimization opportunities, building predictive models, and implementing data-driven user-interaction strategies.

EDUCATION

ESCP Business School (London & Paris)

Master in Management — Finance. Capital markets, financial modeling, corporate valuation, derivatives, structured finance.

2023 — 2026

Beijing International Studies University

BA in Language & Culture (French, Japanese); minor in New Media (Communication University of China); self-taught Psychology (Yale / Coursera certificate).

2019 — 2023

Yanjin Li · geraldineliyanjin@gmail.com · github.com/Yanjin-ai